Тема А.
Принципы построения алгоритмов

Алгоритм — это упорядоченный набор недвусмысленных и выполнимых этапов, расположенных в определенном логическом порядке определяющий некоторый конечный процесс.

Представление алгоритма в виде блок-схемы является промежуточным, так как алгоритм в таком виде не может быть непосредственно выполнен ЭВМ, но помогает пользователю при создании (написании) программы для персонального компьютера (ПК).

Использование блок-схем дает возможность:

• наглядно отобразить базовые конструкции алгоритма;

• сосредоточить внимание на структуре алгоритма, а не на синтаксисе языка;

• анализировать логическую структуру алгоритма;

• преобразовывать алгоритм методом укрупнения (сведения к единому блоку) или детализации – разбиения на ряд блоков;

• использовать принцип блочности при коллективном решении сложной задачи;

• осуществить быструю проверку разработанного алгоритма (на уровне идеи);

• разобрать большее число учебных задач.

Составление блок-схемы алгоритма является важным и в большинстве случаев необходимым этапом решения сложной и большой задачи на ЭВМ, значительно облегчающим процесс составления программ.

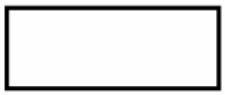
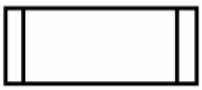
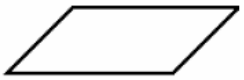
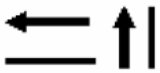
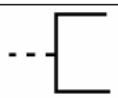
Некоторые, часто используемые обозначения, приведены в табл. 1.

Отдельные блоки алгоритмов соединяются между собой линиями потоков, которые проводятся параллельно внешней рамке чертежа. Направления линий потока сверху вниз и слева направо принимаются за основные и, если линии потоков не имеют изломов, стрелками не обозначаются. Обратные направления линий потока помечаются стрелкой.

«Процесс» (этап вычисления) изображается прямоугольником, внутри которого записывается набор действий. Ромбом изображается «решение», внутри которого осуществляется проверка условия. Ввод исходных данных и вывод результатов изображаются параллелограммами, внутри которых пишутся слова «ввод» или

«вывод» и перечисляются переменные, подлежащие вводу или выводу.

Таблица 1 – Условные графические обозначения символов

Название блока	**Обозначение**	**Название блока**	**Обозна-чение**
Начало или конец алгоритма		Решение	
Процесс (действие или серия действий)		Предопределенный процесс (вспомогательный алгоритм)	
Ввод/вывод данных		Модификация (заголовок цикла)	
Линии потока		Комментарии	

Выделяют три основные структуры алгоритмов:

1. Линейная.
2. Разветвляющаяся (альтернатива «если–то–иначе» или «если–то»).
3. Циклическая (повторение).

Линейная структура – является основной. Она означает, что действия выполняются друг за другом (рис. 1).

Прямоугольник, показанный на рисунке, может представлять как одну единственную команду, так и множество операторов, необходимых для выполнения сложной обработки данных, где ***F1*** и ***F2*** – некоторые команды для соответствующего исполнителя. Команды записываются с помощью операции присваивания.

Присваивание переменной какого-либо значения или присваивание одной переменной значения другой переменной является наиболее часто выполняемым действием в программе, написанной на любом языке программирования. В языке программирования присваивание является операцией и обозначается как «:=». Это означает, что при выполнении этой операции происходит не только присваивание значения определенной переменной, но и возвращается некоторое значение.

Разветвляющаяся структура (ветвление) – это структура, обеспечивающая альтернативный выбор в зависимости от заданного условия. Выполняется проверка условия, а затем выбирается один из путей (рис. 2), где ***P*** – это условие, в зависимости от истинности (Да) или ложности (Нет) которого управление передается по одной из двух ветвей.

Может оказаться, что для одного из результатов проверки (рис. 2). условия ничего предпринимать не надо. В этом случае можно применять только один обрабатывающий блок (рис. 3).

Эта структура называется также, ЕСЛИ – ТО – ИНАЧЕ. Каждый из путей (ТО или ИНАЧЕ) ведет к общей точке слияния, так что выполнение программы продолжается независимо от того, какой путь был выбран.

Циклическая структура (или повторение) предусматривает повторное выполнение некоторого набора действий.

Циклы позволяют записать длинные последовательности операций обработки данных с помощью небольшого числа повторяющихся команд.

Итерационным называется цикл, число повторений которого не задается, а определяется в ходе выполнения цикла. В этом случае одно повторение цикла называется **итерацией.**

Рекурсия – это такая ситуация, когда некоторый алгоритм непосредственно или через другие алгоритмы вызывает себя в качестве вспомогательного. Сам алгоритм при этом называется рекурсивным.

В качестве примера использования рекурсии рассмотрим задачу поиска файлов. Пусть нужно получить список всех файлов, например, с расширением bmp, которые находятся в указанном пользователем каталоге и во всех подкаталогах этого каталога.

Словесно алгоритм обработки каталога может быть представлен так:

1. Вывести список всех файлов, удовлетворяющих критерию запроса.
2. Если в каталоге есть подкаталоги, то обработать каждый из этих каталогов.

Приведенный алгоритм, блок-схема которого представлена на рис. 4, является рекурсивным. Для того чтобы обработать подкаталог, процедура обработки текущего каталога должна вызвать сама себя.

Различают **цикл с предусловием** и **цикл с постусловием.**

Цикл начинается с проверки логического выражения «P». Если оно истинно, то выполняется «F», затем все повторяется снова, до тех пор, пока логическое выражение сохраняет значение «истина». Как только оно становится ложным, выполнение операций «F» прекращается и управление передается по программе дальше.

Так как выражение, управляющее циклом, проверяется в самом начале, то в случае, если условие сразу окажется ложным, операторы циклической части «F» могут вообще не выполняться. Операторы циклической части «F» должны изменять переменную (или переменные), влияющую на значение логического выражения, иначе программа «зациклится» – будет выполняться бесконечно. Рассмотренная циклическая конструкция называется **цикл «пока», или цикл с предусловием** (рис. 5).

Существует и иная конструкция цикла, которая предусматривает проверку условия после выполнения команд, встроенных внутрь цикла. Это **цикл с постусловием** (рис. 6).

Так же существует ***безусловный циклический алгоритм*** (рис. 7), который удобно использовать, если известно, сколько раз необходимо выполнить тело цикла. В этом цикле выполнение и повторение операций происходит от начального значения параметра (счётчика) до его конечного значения с указанным шагом. Если шаг не указан, то его значение полагается равным единице. Выполнение безусловного циклического алгоритма начинается с присвоения переменной **i** стартового значения **in**. Затем следует проверка, не превосходит ли переменная **i** конечное значение **iк**. Если превосходит, то цикл считается завершенным, и

управление передается следующему за телом цикла оператору. В противном случае выполняется тело цикла, и переменная **i** меняет свое значение в соответствии с указанным шагом **di**. Далее, снова производится проверка значения переменной **i** и алгоритм повторяется. Понятно, что безусловный циклический алгоритм можно заменить любым условным (рис.8).

Циклические структуры можно комбинировать одну с другой – как путем организации их следований, так и путем создания суперпозиций (вложений одной структуры в другую).

Схематические изображения нескольких суперпозиций базовых алгоритмических структур представлены на рис. 9–10.

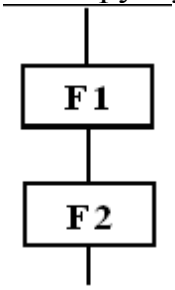


Рис. 1 - Линейная структура

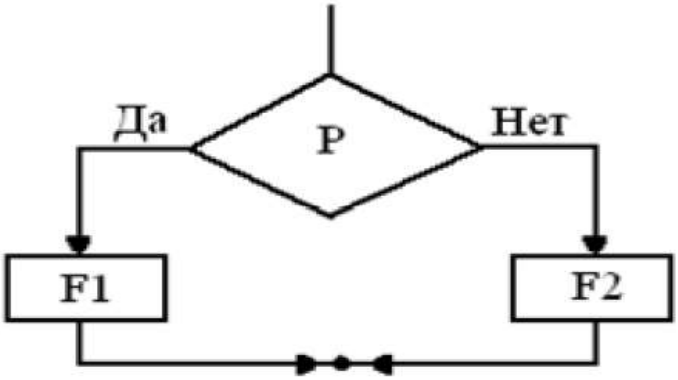


Рис.2 – Ветвление

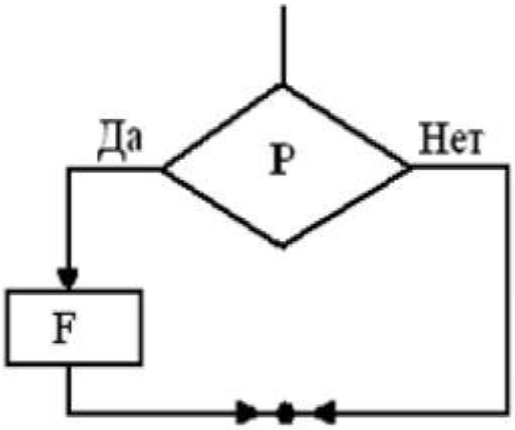


Рис.3 – Неполное ветвление



Рис.4 - Рекурсия

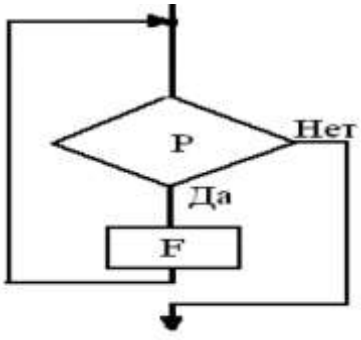


Рис.5 – Цикл с предусловие

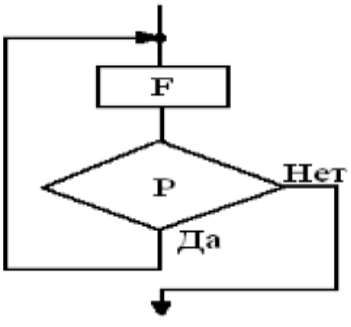


Рис. 6 – Цикл с постусловием

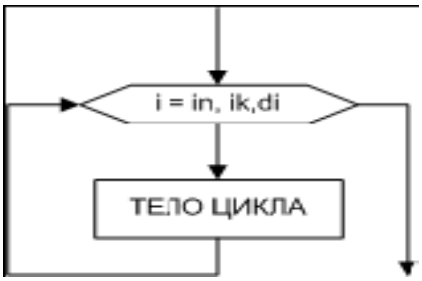


Рис.7 – Алгоритм циклической структуры без условия

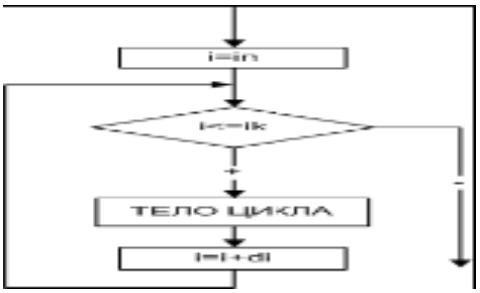


Рис.8 – Условный циклический алгоритм с известным числом повторений

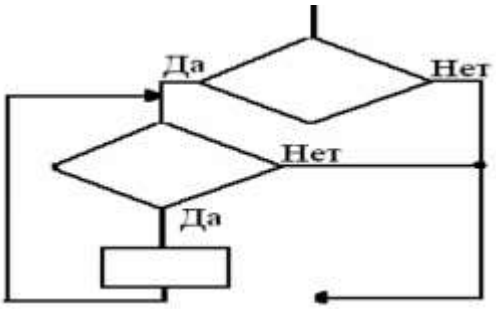


Рис.9 – Алгоритм типа «цикл, вложенный в неполную развилку»

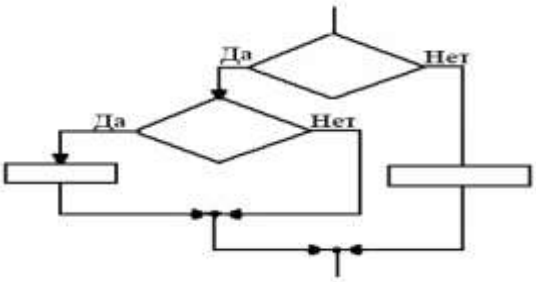


Рис.10 – Алгоритм типа «развилка в развилке»

Пример. Составление алгоритма.

Составить схему алгоритма для нахождения значения системы уравнений вида:

$$x = \begin{cases} \sqrt{y+2}^3, \text{если } y > 7 \\ \sqrt{(2*y-4)^2}, \text{если } y < 7 \end{cases}$$

Блок-схема:

